

Projektdokumentation: Roguelite CLI

Inhaltsverzeichnis

1 Planung	1
1.1 Kanban Board und MoScOw Methode	1
1.2 UML Diagramm und Software Architektur	3
1.3 Testszenarien	3
2 Spiel Anleitung	4

1 Planung

Begonnen hat unser Projekt im Bereich der Objektorientierten Programmierung am 13.04.2026 mit einem Brainstorming für mögliche Themen. Anfänglich ging es um Probleme aus dem IT-Alltag jedoch einigten wir uns schnell darauf ein Spiel zu entwickeln. Unsere Gruppe bestehend aus Marek Stolz, Alexander Behm und Florian Katzenski entschloss sich für ein Text basiertes Spiel, in dem Runden basiert gegen Gegner gekämpft werden kann.

1.1 Kanban Board und MoScOw Methode

Als Projektmanagement Methode haben wir ein Kanban Board gewählt, bestehend aus dem Backlog, was aktuell umgesetzt wird und was abgeschlossen ist. Im Backlog werden Aufgaben definiert und kategorisiert nach „Must have“, „Should Have“ und „Could Have“.

Die „Must have“Funktionen sind absolut notwendig damit wir ein MVP (Minimum viable product) haben. Unser MVP vermittelt die grundlegende Spielidee verfügt aber noch nicht über besondere Features. Die Should haves sind wichtige Features, die jedoch nicht absolut notwendig sind. Zum Beispiel eine Währung um Upgrades kaufen zu können oder besondere Fähigkeiten des Hauptcharakters oder der Monster. Bei unseren „Could haves“ ging es vor allem um die Darstellung des Spiels und bestimmten Fällen, die nicht häufig eintreten („Edge cases“).

Bei der Implementierung haben wir erst die „Must haves“, dann die „Should haves“ und zuletzt die „Could haves“implementiert. Definiert haben wir außerdem die „Won’t haves“, also Dinge die wir nicht in unserem Spiel umsetzen, da Sie zu Zeit aufwendig sind oder zu weit gehen.

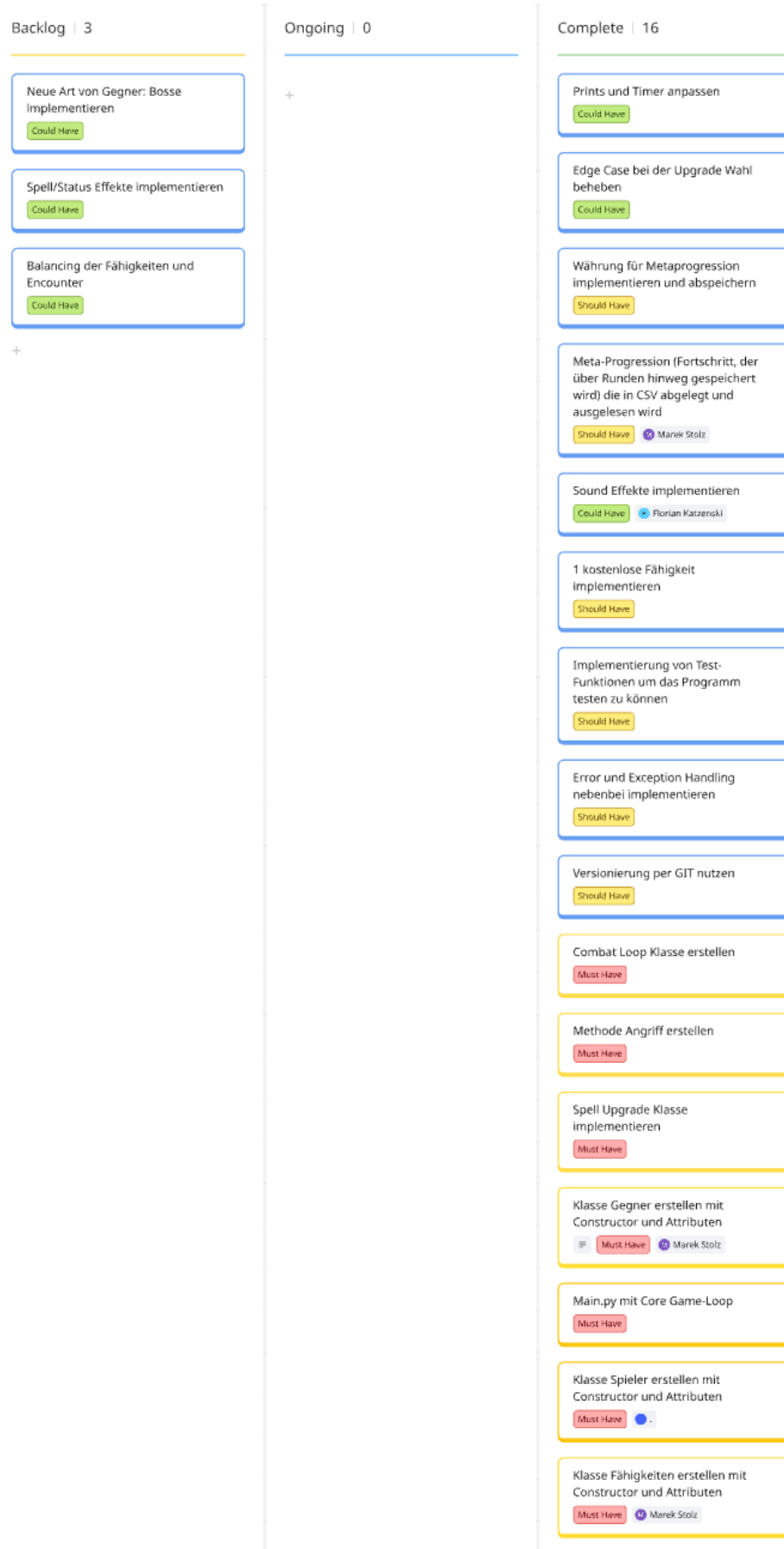


Abbildung 1: Kanban Board

1.2 UML Diagramm und Software Architektur

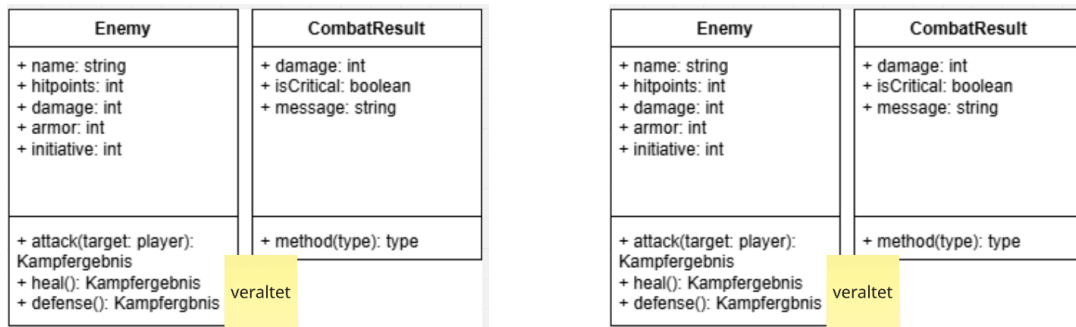


Abbildung 2: Erster Entwurf unseres UML Diagramms
Abbildung 3: Unser UML Diagramm gegen Ende des Projekts

Wir haben uns dazu entschieden die im Kanban Board definierten Features in Klassen umzusetzen und nicht mit Vererbung zu arbeiten. Beispielsweise sind Gegner keine Subklasse des Hauptcharakters. Bei den Methoden haben wir versucht darauf zu achten das „separation of concerns“ Prinzip einzuhalten und Methoden möglichst getrennt voneinander bestimmte Aufgaben zu übergeben.

Zu Beginn war unser UML Diagramm wenig umfangreich und hat nur die Klasse für Gegner und das Kampfergebnis beschrieben. Am Ende des Projekts enthält es alle implementierten Klassen inklusive deren Methoden. Trotzdem ist es keine vollständige Darstellung unseres Source Codes da manche Funktionen nicht enthalten sind.

1.3 Testszenarien

Test ID 7	Beschreibung 7	Startbedingung 7	Eingabe 7	Erwartetes Ergebnis 7	Tatsächliches Ergebnis 7	Success / Failed? 7
T-01 Should not	Happy Path: Gewünschte Fähigkeit wird durch eine Zahl ausgewählt	Fähigkeiten existieren als Funktion Fähigkeiten sind auf eingetragbare Zahlen gemappt	Auswahl: 1	Fähigkeit wird ausgeführt	Fähigkeit wird ausgeführt	Success
T-02	Edge Case: Eine Zahl wird eingegeben ohne das ein Menüpunkt dafür existiert	Fähigkeiten existieren als Funktion Fähigkeiten sind auf eingetragbare Zahlen gemappt	Auswahl: 67	Ausgabe "Dieser Menüpunkt existiert nicht, wähle einen existierenden Menüpunkt"	Ausgabe "Please select a number between 1 and 7"	Success
T-03	Edge Case: Etwas anderes ausser einer Zahl (int) wird eingegeben	Fähigkeiten existieren als Funktion Fähigkeiten sind auf eingetragbare Zahlen gemappt	"Eingabe 1"	Ausgabe: Dieser Menüpunkt existiert nicht. Geben Sie eine Zahl ein, die einem Menüpunkt entspricht.	Ausgabe "Please select a number between 1 and 7"	Success
T-04	Edge Case: Enter ohne Eingabe	Fähigkeiten existieren als Funktion Fähigkeiten sind auf eingetragbare Zahlen gemappt	Auswahl: ""	Ausgabe: Dieser Menüpunkt existiert nicht. Geben Sie eine Zahl ein, die einem Menüpunkt entspricht.	Ausgabe "Please select a number between 1 and 7"	Success
T-05	Edge Case: Spieler wählt eine Fähigkeit (z. B. Magiewangriff), hat aber weniger Mana als die manacost.	Der Mana-stand ist geringer als die Mana-kosten des ausgewählten Angriffs	Auswahl einer gültigen Skill-Nummer (z. B. 2).	System blockiert die Ausführung, zählt kein Mana ab, gibt eine Meldung aus ("Nicht genug Mana") und zeigt das Menü erneut an.	Ausgabe "Not enough Mana!", kein Mana wird abgezogen.	Success
T-06	Edge Case: Der User tippt versehentlich ein Leerzeichen vor oder nach der Zahl.	Eine Eingabe mit dem Menüpunkt "1" wird abgefragt.	Auswahl: "01" statt "1"	Das System ist tolerant und ignoriert das Leerzeichen, die Auswahl "1" wird trotzdem erkannt.	Das System ist tolerant und ignoriert das Leerzeichen, die Auswahl "1" wird trotzdem erkannt.	Success
T-07	Edge Case: User gibt eine Dezimalzahl an	Eine Eingabe wird abgefragt.	Auswahl: "1.5"	Ausgabe: Dieser Menüpunkt existiert nicht. Geben Sie eine Zahl ein, die einem Menüpunkt entspricht.	Ausgabe: Dieser Menüpunkt existiert nicht. Geben Sie eine Zahl ein, die einem Menüpunkt entspricht.	Success

Abbildung 4: Testboard

Während dem entwickeln haben unser Programm regelmäßig getestet. Mehrere Testszenarios haben wir in einem Board notiert. Jeder Test erhält eine ID und eine Beschreibung.

Bei der Beschreibung wird unterschieden in den „happy path “ und den „edge case“ . Ein „happy path “ ist der optimale Weg, bei dem die Eingabe des Users der gewünschten Eingabe entspricht und das Ergebnis der getesteten Funktion auch dem gewünschten Ergebnis entspricht. „Edge cases “ sind die Fälle, bei denen nicht die User Eingabe dem gewünschten Entspricht oder die Ausgabe der Funktion nicht dem von uns gewünschten Entspricht. Von diesen Wegen gibt es in einem Programm extrem viele, weshalb unser Board nur einen Bruchteil davon darstellt.

2 Spiel Anleitung